Storybook-First with Figma

A bidirectional workflow for pixel-perfect atomic design

Keith Sarate · Object Edge

Two failure modes dominate UI work

Visual approximation

A button gets a hex that's *close enough*. Spacing snaps to Tailwind's default scale instead of the Figma value. The font looks right but renders as Inter where Barlow Semi Condensed was specified.

None of this gets caught in code review. It surfaces when a stakeholder opens the page next to the Figma file.

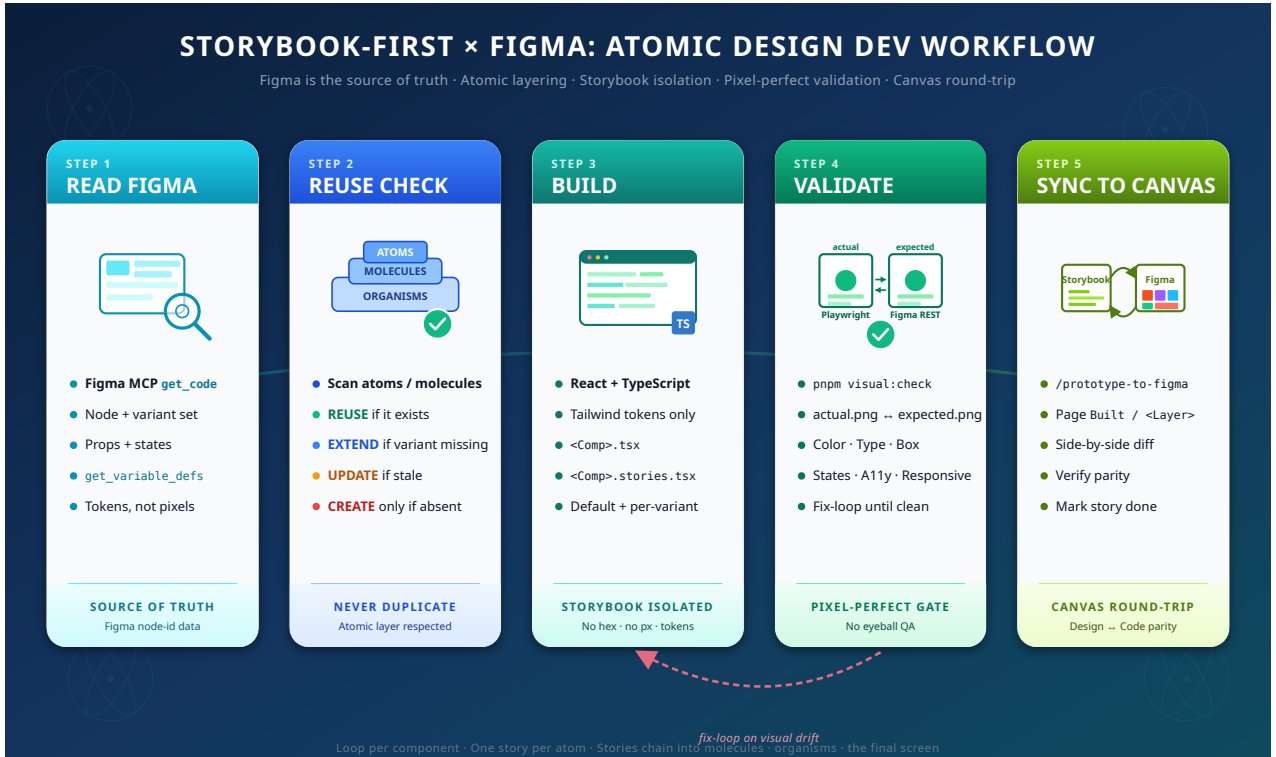
Silent duplication

Every developer builds their own button, their own badge, their own way of truncating text.

Six months in: three `<Button>` components, four definitions of "primary blue."

The proposal, in one sentence

Every UI story follows the same loop —
Read Figma → Reuse Check → Build → Validate → Sync to Canvas —
gated by an automated screenshot diff against the Figma source that can't be eyeballed away.



The five-step loop (per component)

#	Step	What happens
1	Read Figma	`get_code` + `get_variable_defs` via Figma MCP. Tokens, never raw hex.
2	Reuse Check	REUSE / EXTEND / UPDATE / OFF-LAYER / CREATE per node.
3	Build	React + TS + Tailwind. Tokens only. Component + stories per variant.
4	Validate	Playwright @ 2x vs. Figma REST @ 2x. Agent walks the Visual Checklist.
5	Sync to Canvas	Push built component back to Figma on a `Built / <Layer>` page.

Step 0 — what makes it work in brownfield

Runs **once per story**, before any of the 5 steps:

1. Resolve the Figma link
2. Classify target: atom / molecule / organism / screen
3. **Decompose recursively** — find every lower-layer dependency
4. Run the Reuse Check on each node
5. Produce a **bottom-up build plan** (atoms → molecules → organisms → target)

The agent confirms with you if more than three components will be created.

Why Step 0 matters

Most teams install this into projects that have been running for years.

The first story you point it at will frequently target an **organism** or a **screen** with no atoms in `src/components/atoms/` yet.

Step 0 handles it: one dev-story run builds the whole dependency tree, **bottom-up**. Each component still passes through the 5-step loop and visual validation.

The validation gate

`pnpm visual:check` produces two PNGs per variant:

- **`actual.png`** — Playwright screenshots the Storybook story at 2x DPR
- **`expected.png`** — Figma REST API renders the source node at 2x

The agent (multimodal) reads both and walks the **Visual Checklist**:
color · typography · box model · states · a11y

Drift triggers a fix loop. You can't approve it by eye.

How it works under the hood

A **thin execution-discipline layer** on top of standard BMad. No fork, no patched skill.

Three files do the work:

- `_bmad/custom/bmad-dev-story.toml` — the policy (loaded as `persistent_facts`)
- `docs/workflows/storybook-first-with-figma.md` — the operating manual
- `scripts/visual-check.mjs` — ~80 lines, Playwright + Figma REST + Storybook reachability probe

The agent is bound by these rules **regardless of who authored the story**.

Scope guard + preflight

Before any UI story starts, the agent runs two gates baked into `persistent_facts`:

Scope guard — classifies the story as UI or NON-UI. NON-UI stories (backend, infra, docs, schema) skip everything below and fall back to stock BMad.

Preflight check — for UI stories, verifies in order:

1. Figma MCP reachable
2. `FIGMA_TOKEN` set in `.env`
3. `scripts/visual-check.mjs` present
4. Playwright installed
5. `package.json` has `visual:check` script
6. Storybook set up in the project

The three HALT cases

The agent only stops and asks when it can't safely proceed:

1. **UPDATE conflict** — component exists at the right layer but differs from Figma
2. **OFF-LAYER find** — component exists in the codebase but outside the atomic folder structure (common in brownfield)
3. **Explicit scope error** in the original story (e.g. "build all the molecules") — should be split via a discovery pattern

Everything else runs to completion.

What the Figma file needs

The workflow's value is bounded by what's in the file.

- **Variables** for design tokens
- **Atomic-design organization** — frames named or grouped as Atoms / Molecules / Organisms / Screen
- **Components with complete variant sets**

Without these: tokens get derived from inline values, classification becomes guesswork, and Step 0 produces a noisier build plan.

We're honest about this upfront.

Install

From inside the project root (a VS Code terminal works — files land in your working tree):

```
bash <(curl -fsSL https://raw.githubusercontent.com/\
keith-sarate/story-book-first-with-figma/main/install.sh)
```

The script:

- Prompts only for your Figma file key
- Drops **7 files** into the current directory
- Copies `.env.example` → `.env` (file key prefilled, token empty)
- Adds `visual:check` to `package.json` scripts
- Installs Playwright + dotenv via your package manager
- Prints next steps

After install — 1 thing to do

Open `.env` and paste your Figma personal access token in `FIGMA_TOKEN`.

That's it. Invoke `/bmad-dev-story <story.md>` and the agent's preflight will guide you through anything else missing (approve `/mcp`, install Storybook, etc) — you don't have to remember the manual steps.

Once preflight passes, the agent executes **Step 0** (refinement + recursive build plan), confirms with you if many components will be created, then runs the 5-step loop **bottom-up**.

What lands in the target project

```
<target>/
├── .mcp.json                                                  # Figma MCP server
├── .env.example  →  .env                                      # FIGMA_TOKEN, FIGMA_FILE_KEY
├── .gitignore                                                 # appended
├── docs/workflows/storybook-first-with-figma.{md,svg}
├── scripts/visual-check.mjs                                   # Playwright + Figma REST
└── _bmad/custom/bmad-dev-story.toml                           # persistent_facts injection
```

The installer **never modifies BMad core or module files** — only `_bmad/custom/` and project-level paths.

Stories themselves are **per-project** — authored by your PM via `bmad-create-story` from a PRD, with a Figma link in References.

Recap

- Two real failure modes: **visual approximation** + **silent duplication**
- One **five-step loop** per component, with a **screenshot-diff gate** that can't be eyeballed
- **Step 0** makes it work in brownfield — recursive decomposition, bottom-up build
- A **thin discipline layer** over BMad — three files, no fork
- Bounded by the **Figma file quality** — Variables, atomic organization, variant sets

Thank you

Keith Sarate · Object Edge
keith.sarate@objectedge.com

Repository
github.com/keith-sarate/story-book-first-with-figma

Article (PDF)
article/storybook-first-with-figma.pdf